A Functional JAX/Flax Implementation of Next-Concept Modeling for TPU-Oriented Execution

Fabian Balensiefer

November 12, 2026

Abstract

We document an independent JAX/Flax implementation of Next-Concept Modeling (NCM), with explicit codebook state, mask-aware residual quantization, a shared level-wise Transformer and declared distributed placement. Completed milestones add byte-data experiments, atomic resumable checkpoints, frozen-codec prior training, seeded categorical generation and actual-length inference with sparse resampling. On untrained length-512 models at batch 16, local CPU median latency fell from 173.9 to 85.8 ms; this is not a quality-matched or TPU comparison. Four-arm finite-domain studies diagnosed code collisions and separated reconstruction, teacher alignment and dependence objectives. A shared coarse-codebook maintenance package combines deterministic K-means++ warmup, hard-usage EMA replacement, entropy/temperature and commitment schedules, and a separate centroid update group. All 20 new-seed models passed fixed hard-stability and coverage gates; minimum population coarse perplexity was 1.9734 of 2. The reliable teacher-alignment gate still failed. The implementation record reports 44 passing CPU tests, including exact checkpoint continuation and virtual-device checks. These results establish software behavior and a small-domain stabilization result, not useful pretrained model transfer, competitive language modeling or measured TPU performance.

Keywords: next-concept modeling; residual vector quantization; JAX; Flax; TPU; SPMD; hierarchical generation.

1 Introduction

Hierarchical discrete representations provide an interface between a learned codec and a generative prior. Next-Concept Modeling (NCM), introduced by Kumbong et al. [1], combines multiscale residual quantization with coarse-to-fine text generation. This paper concerns an independent systems implementation of that idea rather than a new claim of inventing NCM. The project model has no separate name; `tpu-ncm` is a software package identifier.

Translating a generative design into a compiled distributed program requires decisions that are not captured by its high-level factorization. A quantizer must exclude padding from sufficient statistics, preserve codebook consistency across data shards, and distinguish trainable parameters from moving-average state. A level-wise generator must condition on the cumulative representation available before predicting its current level. Shape-stable control flow must also preserve the semantics of codebook freezing and quantizer dropout.

The implementation uses JAX transformations [2] and the Flax Linen module API [3, 4]. XLA provides the compilation layer [5]. Rust is reserved for a proposed surrounding data and orchestration layer; no Rust component is present in the evaluated snapshot. The implementation is available in the public project repository, version 0.1.0, with this manuscript describing revision `1d600d7`, the November 12, 2026 snapshot [6]. Source hashes identify the files evaluated here independently of later branch updates.

Table 1: Notation shared with the companion manuscript. Code variables I_k are distinct from EMA counts C_{kv}. Prompt u is fixed/empty in the completed finite-domain studies.

Symbol	Meaning
B, L, W	Batch size, finest sequence length, output token vocabulary size
K, n_k, V, D	Levels, level positions, uniform codebook size, code dimension
H, J	Prior hidden width and number of Transformer blocks
I_k, i_{kbj}	Random code level and realized code ID at batch/position (b, j)
E, C, S	Codebook embeddings, EMA counts and residual sums
Q_ψ, d_ψ, p_θ	Deterministic encoder/codec map, token decoder, level prior
$p_s, p_{\theta,\psi}$	Teacher and induced student output distributions
$a, \varphi, \tau_1(a), \beta_1(a)$	Optimizer step, runtime phase, coarse temperature and commitment weight
$\mathsf{H}, \mathrm{TC}, \mathrm{TV}$	Shannon entropy, conditional total correlation, total variation
r, d_s, P_s, R_s	Canonical width, teacher width, calibration map and partial inverse; $r = D = 8$ in the pilot

The contributions are implementation-specific:

1. A mask-aware, common-grid formulation of multiscale residual VQ with explicit codebook embeddings, counts, and residual sums.
2. A shared Transformer that scans over hierarchy levels and keeps current-level labels outside its conditioning path.
3. A functional training transition combining optimizer and EMA state, dynamic phase selection, and explicit `dp`/`fsdp` placement.
4. Executable tests of masking, gradients, state transitions, and virtual multi-device equivalence, plus reproducible data, checkpoint and sampling paths.
5. Actual-length CPU inference measurements and four-arm finite-domain studies of coverage and coarse-codebook stability, with negative alignment results reported alongside the completed engineering gates.

2 Related Work and Scope

Vector-quantized variational autoencoders connect discrete nearest-neighbor assignments to neural encoders and decoders through a straight-through estimator [7]. Residual quantization represents progressively unexplained signal with successive quantizers. SoundStream combines residual VQ with structured quantizer dropout in an audio codec [8]. That result motivates a design pattern; it is not evidence that the same schedule provides useful semantic levels for text.

NCM supplies the principal algorithmic precedent: multiscale residual text codes and a generator that predicts positions within a level in parallel [1]. The published work includes tokenizer/prior staging and conditioning corruption. The repository provides both a joint optimizer interface and staged frozen-codec prior training, clean earlier-level conditioning, and greedy or seeded categorical code selection. It is therefore a systems adaptation, not a reproduction of the published training recipe, checkpoints, or experimental results. The entropy and concept objectives described below are additional project choices. The companion SCALE manuscript [9] develops a conditional cross-size compiler framework. Its completed local mechanism studies share the results below; the full compiler and external pretrained transfer remain proposed.

The generator builds on Transformer attention [10]. Its distribution strategy is also related to the broader use of compiler-based SPMD partitioning exemplified by GShard [11]; this repository does not implement GShard's mixture-of-experts model. JAX's sharding APIs provide the specific execution mechanisms used here [12, 13].

3 Representation and Static Execution Contract

Let $x \in \{0, \dots, W-1\}^{B \times L}$ be token IDs with vocabulary size W. Let $m \in \{0,1\}^{B \times L}$ indicate valid, non-padding positions, and f indicate valid positions routed through byte fallback. The learned code path uses $\ell = m \wedge \neg f$. Throughout the implementation, a true mask entry means *valid*, including arguments named `padding_mask`.

There are K levels, ordered by $1 \le n_1 \le \cdots \le n_K = L$, with code dimension D and V entries per codebook. Every compiled bucket fixes B, K, L, D, V and the complete tuple $(n_1, \dots, n_K)$. Logical codes have lengths n_k, but their common storage is $[K, B, L]$, with structural masks excluding positions $j > n_k$. Invalid code indices are stored as -1. The model does not infer sequence length or fallback geometry during generation; these are caller inputs.

3.1 Mask-normalized grid alignment

Define a resampling operator for nonnegative weights $A \in \mathbb{R}^{T \times S}$:

$$d_{bj}(A, \mu) = \sum_{s=1}^{S} A_{js} \mu_{bs}, \tag{1}$$

$$\mathcal{A}_A(h, \mu)_{bj} = \frac{\sum_{s=1}^{S} A_{js} \mu_{bs} h_{bs}}{\max(d_{bj}(A, \mu), \delta)}, \qquad \delta = 10^{-8}. \tag{2}$$

The destination validity mask is $d_{bj} > 0$. Sources with false masks are zeroed before multiplication. Consequently, a valid output depends only on valid input values, including at partially padded boundaries.

For level k, the fixed matrix A_k performs area pooling from L positions to n_k, padded with zero rows to L. Fractional cell overlaps support lengths that do not divide L. A second fixed matrix U_k interpolates cell centers back to the common length using linear interpolation and constant boundary extension. These are engineering choices in `grids.py`, rather than learned reconstruction operators.

3.2 Text encoder and decoder

Token embeddings feed two residual one-dimensional convolution blocks by default. Each block applies layer normalization, a width-three convolution, GELU, and a dense projection, followed by residual addition and masking. The encoder masks fallback and padding slots before convolution. Its local receptive field and the resampling hierarchy do not by themselves establish that coarse codes contain human-interpretable semantics.

The decoder uses the same residual block form and a linear token-classification head. It accepts a completed latent representation through a separate `decode` method, enabling reconstruction of generated codes without running the encoder again. Its reconstruction objective is token cross-entropy, not an image or audio distortion objective.

4 Multiscale Residual Quantization

Let $z = \mathrm{Encoder}(x, \ell) \in \mathbb{R}^{B \times L \times D}$, with invalid positions zeroed. Set the common-grid cumulative representation $\widehat{z}^{(0)} = 0$. For each level,

$$u_k = \mathcal{A}_{A_k}\big(z - \mathrm{sg}[\widehat{z}^{(k-1)}], \ell\big), \tag{3}$$

$$i_{kbj} = \arg \min_{v \in \{0, \dots, V-1\}} \|u_{kbj} - E_{kv}\|_2^2, \tag{4}$$

$$q_{kbj} = E_{k, i_{kbj}}, \tag{5}$$

$$\widehat{z}^{(k)} = \widehat{z}^{(k-1)} + \ell \odot \mathcal{A}_{U_k}(q_k, \alpha_k). \tag{6}$$

Here α_k combines pooled source validity, structural padding, and any sampled prefix dropout; invalid q_k entries are zero. The source mask ℓ in Eq. (6) also prevents interpolated contributions from filling byte or padding slots. This common-grid subjraction resolves the shape mismatch that would arise from directly subjracting vectors on different level grids.

Squared distances are evaluated as $\|u\|^2 + \|e\|^2 - 2u^\top e$. Hard counts and sums use indexed scatter addition rather than a materialized hard one-hot assignment tensor. The implementation nevertheless computes dense distance and soft-assignment tensors; it does not claim sublinear nearest-neighbor search.

Reconstruction gradients use

$$z_{\mathrm{ST}} = z + \mathrm{sg}[\hat{z}^{(K)} - z]. \tag{7}$$

The forward value is the accumulated quantization, and its derivative with respect to the masked encoder output is the identity. Codebook embeddings are stop-gradient values in the quantizer and prior; the optimizer does not update them through this estimator.

4.1 Explicit EMA state

The complete codebook state is

$$E \in \mathbb{R}^{K\times V\times D}, \quad C \in \mathbb{R}^{K\times V}, \quad S \in \mathbb{R}^{K\times V\times D}. \tag{8}$$

For each batch, sufficient statistics are

$$c_{kv} = \sum_{b,j} \alpha_{kbj} \mathbb{1}[i_{kbj} = v], \tag{9}$$

$$s_{kv} = \sum_{b,j} \alpha_{kbj} \mathbb{1}[i_{kbj} = v] u_{kbj}. \tag{10}$$

After data-parallel aggregation, an eligible level updates by

$$C'_{kv} = \gamma C_{kv} + (1-\gamma) c_{kv}, \tag{11}$$

$$S'_{kv} = \gamma S_{kv} + (1-\gamma) s_{kv}, \tag{12}$$

$$E'_{kv} = \frac{S'_{kv}}{C'_{kv} + \epsilon}. \tag{13}$$

Defaults are $\gamma = 0.99$ and $\epsilon = 10^{-5}$. Initialization draws E from a zero-mean normal distribution scaled by $D^{-1/2}$, uses $C = 1$, and sets $S = E(C+\epsilon)$. A level with no valid assignments preserves its entire old state. For a nonempty eligible level, Eq. (13) applies to all entries, including unused codes. This is the retained legacy configuration. The opt-in maintained coarse path below supersedes its coarse initialization and update policy without changing the fine-book equations.

4.2 Deterministic maintained coarse codebook

With maintenance enabled, the initial encoder's valid pooled coarse latents are collected from a configured number of initial training batches before updates. K-means++ seeding [14] and fixed-count Lloyd refinement use an experiment-seeded JAX key. Degenerate K-means++ seeds are sampled from valid latents; small seeded perturbations prevent coincident seeds when observations repeat. The completed protocol used two batches, 128 pooled samples, 20 iterations and initialization noise 10^{-4}. This is a warmup of the initial encoder's geometry, not a fit on a pretrained semantic manifold.

A separate hard-usage statistic U_v, distinct from the ordinary EMA count C_{1v}, follows

$$U'_v = \gamma_U U_v + (1-\gamma_U) c_{1v}. \tag{14}$$

After the grace interval, eligible codes below a configurable count threshold are replaced with a uniformly sampled valid current-batch latent plus seeded Gaussian noise. Their usage and centroid count/sum statistics are reset consistently. Counts use global assignments, not per-shard frequencies. Candidates and perturbations are computed on every eligible step, then selected by masks, so all four arms execute the same graph even when no code is dead. All-padding batches skip the entire state transition. Frozen coarse phases preserve its embeddings, counts, sums and usage while the global training clock can advance. Initialization flags prevent any refit on resume.

For $s(a) = \min(1, a/A)$, the coarse soft-assignment temperature and commitment weight use independently configurable linear schedules

$$\tau_1(a) = \tau_{\text{start}} + s(a)(\tau_{\text{end}} - \tau_{\text{start}}), \quad \beta_1(a) = \beta_{\text{start}} + s(a)(\beta_{\text{end}} - \beta_{\text{start}}). \tag{15}$$

Separate horizons may be supplied for each schedule. Hard nearest-neighbor quantization remains the forward path; temperature controls the differentiable soft statistics, not a Gumbel sampler. The coarse commitment replaces that level's baseline weight within the original valid-component normalization. An optional extra loss $-\lambda_{\text{entropy}}\mathsf{H}(q_{1,\text{marginal}})$ uses masked soft assignments. It encourages marginal spread but does not imply low conditional TC or uniform hard usage.

Neural parameters retain their Adam group. Coarse embeddings use a separate stateless SGD group on the centroid displacement pseudo-gradient $g_E = E_1 - E_1^{\text{EMA}}$, giving

$$E_1' = E_1 + \eta_{\text{coarse}}(E_1^{\text{EMA}} - E_1). \tag{16}$$

The setting `coarse_codebook_lr_scale`=0.25 is a quarter of the EMA centroid proposal, not 0.25 times the neural Adam learning rate. Fine embeddings keep their ordinary EMA update. Because the maintained embedding can lag its centroid target, it need not equal $S/(C + \epsilon)$ at every step. The checkpointed optimizer tree stores neural state under `neural` and coarse initialization, random key, usage, replacements and schedule progress under `coarse`; codebook (E, C, S) remains explicit. Disabled maintenance retains the legacy state schema. Canonical hashing sorts mapping keys, and mid-replay continuation is tested bit-exactly on the same backend.

5 Shared Level-Wise Transformer Prior

At the start of level k, the prior resizes the cumulative earlier-level representation and projects it to hidden width H:

$$h_{kbj}^{(0)} = W_c \mathcal{A}_{A_k}(\hat{z}^{(k-1)}, \ell)_{bj} + b_c + e_k^{\text{level}} + e_j^{\text{pos}}. \tag{17}$$

A stack of pre-normalized attention and feed-forward blocks produces $[B, L, V]$ logits. The stack, context projection, positional table, and output head share parameters across levels; the level-embedding table distinguishes their roles. Attention allows all valid queries within the current level to interact. No query embeds a current-level target code.

For fixed geometry, the corresponding prior has the form

$$p_\theta(i_{1:K} \mid \mathcal{G}) = \prod_{k=1}^{K} \prod_{b,j:\alpha_{kbj}=1} p_\theta(i_{kbj} \mid \hat{z}^{(k-1)}, k, \mathcal{G}), \tag{18}$$

where $\mathcal{G}$ contains lengths and masks. Dependencies between positions can be represented through deterministic query interactions and earlier levels, but code selections within a level do not condition on already selected current-level codes.

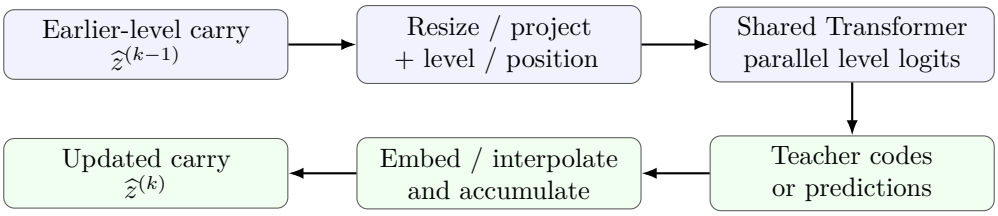


Figure 1: One level of the scanned prior. Current-level teacher codes are introduced after the logits are computed. The updated carry is the input to the next level; predictions may be greedy or categorical samples. Fallback features enter the final decoder separately.

Teacher forcing computes the logits before adding the current ground-truth code vectors to the carry. In free-running mode it adds its own greedy argmax predictions instead; a separate sampling interface supports seeded categorical selection, temperature and optional top-k. A traced scalar boolean selects these paths while target arrays retain a fixed dummy shape. There are K sequential Transformer evaluations for generation, not $\sum_k n_k$ token-autoregressive calls. The implementation also scans levels during training; it does not implement a single concatenated, all-level training attention call.

6 Training Objective and Gradient Boundaries

The implemented objective is

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \beta\mathcal{L}_{\text{commit}} + \lambda_H\mathcal{L}_H + \lambda_C\mathcal{L}_{\text{concept}} + \lambda_{\text{prior}}\mathcal{L}_{\text{prior}}. \tag{19}$$

Default coefficients are $\beta = 0.25$, $\lambda_H = 0.01$, $\lambda_C = 0$, and $\lambda_{\text{prior}} = 1$. Coefficients are configuration choices, not experimentally optimized settings.

6.1 Reconstruction and commitment

Let d_ψ be the token decoder distribution. Reconstruction includes every valid token, including slots using byte features:

$$\mathcal{L}_{\text{recon}} = -\frac{\sum_{b,j} m_{bj} \log d_\psi(x_{bj} \mid \widetilde{z})}{\max\left(1, \sum_{b,j} m_{bj}\right)}. \tag{20}$$

The fallback-selected representation $\widetilde{z}$ is defined in Section 7.2. Commitment is a mean over all valid scalar residual components, rather than an unweighted mean of per-level means:

$$\mathcal{L}_{\text{commit}} = \frac{\sum_{k,b,j} \alpha_{kbj} \|u_{kbj} - \text{sg}[q_{kbj}]\|_2^2}{\max\left(1, D\sum_{k,b,j} \alpha_{kbj}\right)}. \tag{21}$$

6.2 Differentiable marginal entropy

An entropy computed only from hard argmin counts has zero encoder derivative almost everywhere. The implementation therefore uses soft assignment masses for the regularizer, while retaining hard assignments for EMA and diagnostics:

$$\pi_{kbjv} = \text{softmax}_v\left(-\|u_{kbj} - E_{kv}\|_2^2/\tau\right), \tag{22}$$

$$\bar{c}_{kv} = \sum_{b,j} \alpha_{kbj}\pi_{kbjv}, \qquad p_{kv} = \frac{\bar{c}_{kv}}{\max(\sum_{v'} \bar{c}_{kv'}, 10^{-8})}, \tag{23}$$

$$\mathcal{L}_H = \sum_{k=1}^{K} w_k \sum_{v=0}^{V-1} p_{kv} \log(p_{kv} + \epsilon_H). \tag{24}$$

The default temperature is $\tau = 1$, $\epsilon_H = 10^{-5}$, and $w_k = K - k + 1$. Empty levels contribute zero. This is a smoothed, soft-assignment estimate of negative marginal entropy; it need not equal the entropy of hard codes. Optimizing it can increase uncertainty without increasing semantic usefulness. Experiments must report hard code usage, reconstruction, and prior prediction jointly.

6.3 Optional concept consistency

The coarse feature is the first level's code vectors interpolated onto the common grid. A learned dense projection P maps the completed straight-through representation into the same feature dimension. Over the intersection $\mathcal{M}$ of valid coarse and learned-path positions,

$$\mathcal{L}_{\text{concept}} = \frac{\sum_{(b,j)\in\mathcal{M}} \left[1 - \cos_\delta \left(\mathcal{A}_{U_1}(q_1, \alpha_1)_{bj}, P(z_{\text{ST}})_{bj}\right)\right]}{\max(1, |\mathcal{M}|)}. \tag{25}$$

The implementation stabilizes each feature norm by $\sqrt{\|g\|^2 + 10^{-8}}$. When $\mathcal{M}$ is empty the loss is zero. The coarse hard-code path is non-differentiable; gradients train the projection and can reach the encoder through the fine straight-through representation. Cosine agreement is an auxiliary hypothesis, not a measured semantic property.

6.4 Prior loss and joint optimization

The prior is trained against aligned current-level code labels:

$$\mathcal{L}_{\text{prior}} = -\frac{\sum_{k,b,j} \alpha_{kbj} \log p_\theta(i_{kbj} \mid \widehat{z}^{(k-1)}, k, \mathcal{G})}{\max\left(1, \sum_{k,b,j} \alpha_{kbj}\right)}. \tag{26}$$

"Next" refers to the next hierarchy level, not a one-position shift within that level. A separate shifted token cross-entropy helper exists for an optional token-causal extension; it is not the NCM prior objective.

The baseline optimizer holds encoder, decoder, projection and prior parameters; the maintained path separates neural and coarse-centroid groups. However, the hard indices and prior codebook gathers are stop-gradient values. Thus prior CE trains the generator, not the encoder through code identities. The codec still changes through reconstruction and auxiliary terms, so the prior sees moving targets in joint training. Frozen-codec prior training is implemented and was used in the text and finite-domain runs. Its checkpoint includes the frozen codec identity; joint training remains an available interface rather than the protocol for those runs.

7 Runtime Phases and Byte Routing

7.1 Phase-dependent EMA eligibility

Let φ be a scalar runtime integer. Level k, using one-based indexing, is eligible for EMA if

$$e_k = \mathbb{1}[\varphi = 0 \ \vee \ k > K - 2]\, \mathbb{1}\left[\sum_v c_{kv} > 0\right]. \tag{27}$$

If $e_k = 0$, all three tensors E_k, C_k, S_k retain their old values. For $K \le 2$, all levels remain eligible. The final phase freezes coarse *codebook state*, not encoder or decoder parameters.

During joint-phase training, a Bernoulli selector with configured probability p_d chooses whether to sample a per-example prefix cutoff uniformly from $\{1, \ldots, K\}$; otherwise all levels are retained. The mask enforces the cutoff without changing array sizes. For $\varphi \neq 0$, the retained depth is always K. The scalar predicate changes values inside the executable rather than changing a static Python argument. If the finest level is dropped, the prior still needs the original learned-path source mask; it must not infer that mask from the dropped level alone.

Table 2: Implemented placement policy. P denotes a JAX partition spec; matrix/tensor axes are chosen by a deterministic divisibility heuristic.

State or input	Placement
Input batch, shape $[B, L]$	$P(\texttt{dp}, \text{None})$; global B is divisible by mesh dp size
Matrix/tensor parameter	Largest divisible axis sharded over `fsdp`; replicated over `dp`
Biases and norm scales	Replicated
Optimizer moments	Same shape-based placement rule as corresponding parameter leaves
Optimizer counters	Replicated
Codebooks E, C, S	Replicated over both mesh axes
Phase, random key, diagnostics	Replicated

7.2 Fixed-shape fallback representation

Let $b_{bj} \in \{0, \ldots, 255\}$ be a byte ID supplied by the data producer and $F \in \mathbb{R}^{256\times D}$ its embedding table. The decoder input is

$$\widetilde{z}_{bj} = m_{bj}\left[f_{bj}F_{b_{bj}} + (1 - f_{bj})z_{\mathrm{ST},bj}\right]. \tag{28}$$

This selection occurs in the compiled graph. Fallback slots are excluded from the learned encoder inputs, quantizer assignments, and EMA statistics; they remain valid reconstruction targets.

The interface requires the input pipeline to expand unsuitable UTF-8 regions into aligned byte slots before bucketing and to assign compatible decoder labels. Arbitrary token vocabulary IDs do not determine those bytes by themselves. Exact literal recovery requires retaining the byte IDs and masks in the external representation; an embedding followed by a neural decoder is not inherently lossless. A mixed token/byte serialization format, UTF-8 assembly, and a policy for selecting unsuitable regions remain unimplemented.

8 Functional Execution and SPMD Placement

The train state is a pytree containing neural parameters Θ, optimizer state Ω, including maintained coarse buffers when enabled, codebook state (E, C, S), and a step counter. The transition has the form

$$(\Theta', \Omega', E', C', S', a'; \mathcal{D}) = \mathrm{Step}(\Theta, \Omega, E, C, S, a; \mathcal{B}, \varphi, \xi), \tag{29}$$

where ξ is a random key and $\mathcal{D}$ contains diagnostics. Targets and gradients are computed from the old state, the optimizer updates neural parameters, and codebook maintenance updates the explicit codebooks once. An all-padding global batch preserves optimizer state, parameters, EMA state, and the step counter. It is a semantic skip; this branch does not guarantee that all otherwise-computed arithmetic is avoided by the compiled executable.

Quantization uses `jax.lax.scan`. The prior uses a lifted Flax scan that broadcasts parameters and splits dropout keys [4]. The train function has explicit input and output `NamedSharding` contracts [12]. The mesh axes are `dp`, for data parallelism, and `fsdp`, for parameter partitioning.

8.1 Reduce sufficient statistics exactly once

The end-to-end step consumes *global arrays*. A sum over the global batch dimension has global semantics; XLA supplies the required communication for its sharded execution. That path does not introduce an explicit axis-named `psum`. In contrast, a local-array function under a bound manual mapping needs its local statistics summed over the bound data axis [13]:

$$c_{kv}^{\mathrm{global}} = \sum_{d\in\mathrm{dp}} c_{kv}^{(d)}, \qquad s_{kv}^{\mathrm{global}} = \sum_{d\in\mathrm{dp}} s_{kv}^{(d)}. \tag{30}$$

The low-level quantizer and EMA functions expose an optional `axis_name` for this case. If the quantizer has already reduced its outputs, EMA must not reduce them again. A named mesh alone does not bind a collective axis inside an ordinary global-array JIT function.

Mask-aware means similarly reduce numerators and denominators, rather than averaging means of unequally padded shards. All replicas then apply the same EMA equations to the same global statistics. Codebooks are not sharded along V; distributed nearest-neighbor lookup would require additional design.

8.2 Multi-host boundary and Rust tooling

The repository includes a helper to construct global batches from process-local data. It requires distributed initialization before mesh creation and coordinated calls across hosts. The example initializes a complete small host state; scalable checkpoint restoration and multi-host initialization have not been validated.

A proposed Rust layer would ingest corpora, expand fallback bytes, form length buckets, validate IDs/masks, and produce shard files and metadata. It could also coordinate runs and aggregate evaluation outputs. The neural forward passes, gradients, optimizer, EMA, and collectives remain in JAX/XLA. This separation defines an integration boundary; it is not a claim that a Rust pipeline has already been implemented or benchmarked.

9 Complexity and Potential Bottlenecks

The padded level storage supports one scan body shape, at a cost. Dense nearest-neighbor distances require $O(KBLVD)$ arithmetic; retained prior logits use $O(KBLV)$ elements. The implementation also constructs dense resampling matrices with $O(KL^2)$ elements, and its dense alignment contractions can require $O(KBL^2D)$ work. Masks enforce correctness but do not by themselves eliminate these operations.

For J dense-attention Transformer blocks with hidden width H and fixed feed-forward expansion, padded attention/MLP work is approximately

$$O\Big(KJB(L^2H + LH^2) + KBLHV\Big). \tag{31}$$

This is an operation-count model of the source layout, not a measurement of the compiler's memory schedule or realized device throughput. The implemented optional actual-length inference path instead uses a static Python level loop with the same parameters and attention work proportional to $JB\sum_k(n_k^2H + n_kH^2)$. Sparse area-pooling/interpolation plans remove the dense resampling layout on this path; dense nearest-neighbor distances remain. Greedy outputs are checked against the padded reference. Different stochastic key-consumption layouts need not give identical sampled sequences.

Two simple useful-position fractions are

$$\eta_{\text{levels}} = \frac{\sum_k n_k}{KL}, \qquad \eta_{\text{attention}} = \frac{\sum_k n_k^2}{KL^2}. \tag{32}$$

For the example schedule $(2, 4, 8)$, these are approximately 0.583 and 0.438, respectively, before content padding. They describe layout utilization, not measured speedups. Replicated codebooks cost $O(KVD)$ memory per device; global sufficient-statistic aggregation communicates arrays of sizes $[K, V]$ and $[K, V, D]$. Parameter sharding can introduce additional communication. Reducing sequential prediction decisions to K does not imply a corresponding wall-clock improvement over token-autoregressive models.

10 Software Validation

The implementation's November 12 validation record reports **44 passed**, with zero failures; manuscript revision did not change model code. The environment and evidence are detailed in the completed experiments section. Counts include parameterized pytest cases, not only Python test functions. The suite covers VQ/loss behavior, deterministic distinct-centroid initialization, dead-code replacement and hard usage, generation, benchmark implementations, coverage/dependence controls, canonical checkpoint hashes and exact continuation. A newly collected test inventory agrees with the recorded 44-case suite.

10.1 Distributed comparison protocol

A fresh interpreter exposes four virtual CPU devices using the setting

```
XLA_FLAGS=--xla_force_host_platform_device_count=4
```

The integrated test uses $B = 4$, $L = 4$, $(n_1, n_2, n_3) = (1, 2, 4)$, $D = 4$, $V = 4$, token vocabulary $W = 16$, and a one-block prior with width 8 and two heads. Its four examples contain 4, 3, 1, and 0 valid positions. These unequal counts exercise global normalization under padding.

An unsharded global-array step and a 2×2 `dp`/`fsdp` step start from the same initialized state and random keys. Every returned parameter, optimizer leaf, EMA leaf, step count, and metric is compared with absolute and relative tolerance 3×10^{-5}. The test separately checks that all codebook leaves replicate and that at least one parameter and optimizer leaf is actually sharded. The comparison uses Adam with learning rate 10^{-3} and $\epsilon_{\text{Adam}} = 10^{-5}$ to avoid amplifying roundoff in near-zero gradients. It does not assert bitwise equivalence between reduction orders.

A second integrated virtual-device test enables coarse maintenance with $V = 2$, checking its complete state against the same global reference. Additional local-array `pmap` checks sum artificial EMA statistics and compare them against a direct global update. They also compare quantizer counts, sums, soft counts, and commitment numerators/denominators across the local/global paths with absolute tolerance 10^{-6} and NumPy's default relative tolerance. This manual path validates sufficient-statistic reduction, not a complete independently implemented local-array training loop.

10.2 Synthetic training execution

The example executes three AdamW steps on a fixed synthetic batch with $B = 2$, $L = 8$, $(n_1, n_2, n_3) = (2, 4, 8)$, $D = 16$, $V = 16$, and $W = 256$. The prior uses two blocks, width 32, and four heads; concept loss weight is 0.1. The final step switches to the final-two-level phase. The example completes with finite reported objectives. It is an execution check with a reused toy batch, not a corpus experiment, convergence study, or held-out generalization result.

11 Completed CPU Experiments

11.1 Evidence snapshot and completed engineering milestones

All results in this section describe the implementation at revision `1d600d7`, dated November 12, 2026, package version 0.1.0 [6]. The recorded stack was Python 3.12.16, JAX/jaxlib 0.10.2, Flax 0.12.8, Optax 0.2.8 and NumPy 2.5.4, on macOS 26.6.2 arm64 CPU, with float32 arrays. The CPU model was not recorded; no TPU hardware was available. The validation record reports 44 passing pytest cases, including parameterized runs, exact checkpoint continuation and two virtual four-device CPU sharding checks. Test runtime is not an algorithmic benchmark.

The completed engineering path includes UTF-8 byte loading and chunking, configuration/-data hashes, atomic checkpoint replacement, step-addressed random batches, a frozen-codec

Table 3: Synchronized single-device CPU median generation latency (ms), batch size 16, float32, five warmups and 30 repetitions. NCM includes byte decoding; AR uses a KV cache. Length 8 uses trained toy models of width 16 and one block; lengths 128/512 use untrained width-64, two-block shape checks. These are not quality-matched speedups or TPU measurements.

Bytes	Scope	Padded NCM	Actual-length NCM	Cached AR
8	Trained toy	0.489	0.403	0.937
128	Untrained shape	26.877	14.390	39.588
512	Untrained shape	173.887	85.831	299.951

Table 4: Completed finite-domain protocols. Known seeds are repeated initializations, not new independent confirmations. Every four-arm follow-up fits 32 student models, of which 20 use newly reserved student seeds; the teacher, domain and sampled datasets remain fixed.

Protocol	Known-seed checks	Evaluation seeds	Models
Initial pilot	–	0,1,2	9
Decoder replay	0,1,2	3,4,5,6,7	32
Balanced codec	0,4,7	8,9,10,11,12	32
Coarse maintenance	4,8,10	13,14,15,16,17	32

prior, seeded categorical code sampling with temperature and optional top-k, and byte decoding. Output length remains a caller input; there is no prompt-conditioning, learned-length or EOS interface. The separate 26-byte training and 13-byte validation fixture reached 100% training-byte reconstruction by step 100 of a 500-step run. Validation CE increased from 1.949 nats at step 100 to 3.550 at step 500: this was overfitting, not a successful held-out language result. The historical codec/prior logs are now archived in `reports/text-experiments/`; their checkpoint hashes match the CPU benchmark's source identities. Those records are distinct from the finite-domain teacher studies below.

11.2 CPU inference measurements

The benchmark timed lowered/compiled functions separately from warmed inference, kept timed inputs on-device and waited for every output with `jax.block_until_ready`. Sampling used temperature 1.0; host text conversion and checkpoint I/O were excluded. NCM byte decoding and all cached AR serial steps were included. With batch 16 and untrained length-512 models, actual-level execution reduced median latency from 173.9 to 85.8 ms. The trained length-8 fixture stored 21,328 codec parameters, 4,032 prior parameters and 768 codebook embedding elements, whereas AR had 11,904 parameters. Neither parameter count nor quality was matched. Longer shape checks used $D = 32, V = 64, H = 64, J = 2$, four attention heads and levels $(L/4, L/2, L)$; they were not trained on text.

Compiler live-buffer estimates, process-lifetime counters where supported, compilation times and 18 separate profiling traces were recorded. Per-case measured peak device memory was unavailable; compiler estimates are not peak memory measurements. These observations establish a local implementation improvement, not TPU acceleration or a quality-matched model speedup.

11.3 One-teacher finite-domain protocol

Each domain input is a length-4 sequence over A/B (16 possibilities). A locally trained causal byte Transformer has width 16, two heads and one block; it is not an external pretrained language model. Its source distribution mixed probability $1 - \rho$ equally over AAAA/BBBB with $\rho = 0.25$ uniformly over all 16 sequences. After 400 teacher updates, its A/B logits were normalized

at every prefix to define an exactly enumerable law p_s. Teacher seed 2026 and its checkpoint remained fixed in every protocol.

Training, validation and calibration comprised 512, 512 and 256 independently sampled sequence draws, respectively. They shared the finite support and could repeat identical sequences across splits; there was no held-out domain or larger-teacher evaluation. A sign-fixed PCA interface fitted on calibration features mapped normalized causal teacher features to $r = D = 8$. Student codebooks had $K = 2$, $(n_1, n_2) = (1, 4)$, $V = 2$, and nominal capacity 5 bits; student priors used width 16, two heads and one block. The byte vocabulary was $W = 256$, with no fallback side channel in these studies. The AR teacher used BOS ID 256 on its input only.

Arms were R (reconstruction/commitment/utilization), A (R plus representation alignment), D (R plus dependence) and AD (both). Paired initial states, minibatch order, architecture and update counts were fixed within each seed. Every student used 400 codec updates and 400 frozen-codec prior updates, global batch 64 and Adam learning rate 0.003. The three four-arm follow-ups additionally used 200 decoder-only replay updates at the same rate. Prior parameters and code assignments stayed fixed during replay. Shared teacher/calibration preparation was charged equally; zero auxiliary coefficients still executed the common student objective graph. Equality of compiler-estimated student step FLOPs is not equality of measured total hardware work or a compute-saving claim.

11.4 Metrics and hard-code diagnostics

For a teacher output law p_s and induced student law $p_{\theta,\psi}$, the reported output error is

$$\mathrm{TV}(p_s, p_{\theta,\psi}) = \tfrac{1}{2} \sum_x |p_s(x) - p_{\theta,\psi}(x)|. \tag{33}$$

Every codeword is enumerated and passed through the deterministic byte-argmax decoder. An extra output bin retains probability outside the A/B domain. Missing teacher support is $\sum_{x:p_{\theta,\psi}(x)=0} p_s(x)$. If this is positive, forward KL is infinite; JSON reports a null KL together with the missing mass, not a smoothed finite substitute.

Hard conditional dependence is evaluated under the teacher-induced code law $P_{s,\psi}(i) = \sum_{x:Q_\psi(x)=i} p_s(x)$:

$$\mathrm{TC}(I_k \mid I_{<k}) = \sum_{j=1}^{n_k} \mathsf{H}(I_{k,j} \mid I_{<k}) - \mathsf{H}(I_k \mid I_{<k}). \tag{34}$$

Exact entropies are checked against an independent factorized-prior KL calculation. The training surrogate instead enumerates a mixture of soft assignment products; its gradients and numerical value do not certify the hard code law. Enumeration is capped at 4096 codewords.

Byte reconstruction averages correct valid positions; sequence reconstruction requires every valid byte to match. These are separate metrics. Coarse perplexity is $\exp \mathsf{H}(p_v^{\mathrm{hard}})$, using hard assignments; the theoretical maximum is V. Population values weight all domain inputs by p_s; monitored minibatch counts use actual global training assignments. Teacher-forced code CE conditions on true earlier codes, whereas free-prefix CE uses greedy earlier predictions. Their difference diagnoses exposure error; neither is directly comparable to an AR next-byte perplexity, and paired free-running byte accuracy is not a general quality score for unconditional samples.

11.5 Completed controls, negative results and coverage repair

The initial three-seed pilot passed its fixed AD-versus-A TV/dependence/fidelity gates. Mean hard conditional TC fell from 0.4550 to 0.1368 nats. That comparison lacked a D-only control; most TV gain came from one seed, and seed 0's AD model missed 5.12% of teacher probability. It therefore did not establish a reliable alignment contribution or full SCALE.

Table 5: Mean exact output total variation (TV; lower is better). Columns R, A, D, and AD denote reconstruction, alignment, dependence, and their combination. Only the listed student seeds enter each row; the first row has no decoder replay, later rows include 200 replay updates. Different rows use different seed sets and interventions, so row-to-row means are not causal quality comparisons.

Study	Student seeds	R	A	D	AD
Initial pilot	0,1,2	0.3316	0.3339	–	0.1744
Decoder replay	3,4,5,6,7	0.2417	0.1453	0.2472	0.1577
Balanced codec	8,9,10,11,12	0.4257	0.2370	0.5229	0.2468
Coarse maintenance	13,14,15,16,17	0.1381	0.1318	0.1392	0.1342

The four-arm decoder-replay protocol reserved student seeds 3–7. Its coverage and reliable-alignment gates both failed. In AD seed 4, 16 inputs occupied only 10 codewords. The deterministic code-to-sequence reconstruction ceiling $\sum_i \max_{x:Q_\psi(x)=i} p_s(x)$ was 89.80%; replay achieved 87.87% sequence reconstruction and still missed 6.63% of teacher mass. More decoder updates could not recover distinctions already merged into the same code.

The balanced-codec protocol reserved seeds 8–12. Codec reconstruction and alignment sampled uniformly over distinct observed training sequences; self-normalized training-frequency importance weights kept soft dependence aimed at empirical teacher mass. Prior training retained the original empirical frequency distribution. No population probabilities or validation examples selected repair targets. All five new AD runs retained 16 encodings and achieved 100% sequence reconstruction and complete output support, already before extra replay. The known failing seed 4 also recovered under the changed sampler. The reliable-alignment gate nevertheless failed: AD beat D on only 3/5 new seeds, and A had slightly lower mean TV than AD. Thus the earlier dependence benefit was not uniform across samplers and initializations.

11.6 Coarse-codebook stabilization

The maintained protocol reserved seeds 13–17; seeds 4, 8 and 10 were known-case checks. It combined two initial encoder batches, 20 K-means++/Lloyd iterations, independent hard-usage EMA decay 0.9, threshold 1 assignment per global batch, 25-step dead-code grace, reset count 2 and replacement noise 0.01. It added coarse entropy coefficient 0.05, temperature 2.0 to 0.25 and commitment 0 to 0.25 over 200 steps, and centroid step scale 0.25. Fine-book EMA and neural Adam rates were unchanged. All four arms executed the same maintenance logic. The individual components were not separately ablated.

The fixed stability gate required both codes active, population coarse perplexity at least 1.5, no more than 5% collapsed minibatches after the warmups, complete teacher output support and sequence reconstruction at least 98%. All 20 new-seed models (five seeds times four arms) passed. Minimum population perplexity was 1.9734 out of 2; each model recorded zero collapsed steps over 200 post-annealing observations. Both codes remained active and output support was preserved. AD minimum sequence reconstruction was 99.07%; the other three arms reached 100% on the new seeds. All 32 runs, including known-case checks, had matching compiler-estimated per-step FLOPs: 10,564,359 for codec training, 6,996,102 for prior training and 5,837,030 for decoder replay. Initialization used 128 valid pooled samples per model and was shared per seed, with its budget charged equally.

The reliable teacher-alignment gate was distinct: AD had to beat D by mean TV at least 0.02 and win on at least 4/5 new seeds, while preserving coverage. It won 4/5 but its mean improvement was only 0.00498, so that gate still failed. Stable codebooks are an engineering achievement, not evidence of cross-size transfer, reduced total adaptation compute or general

language-model quality. All gates were saved locally before their study; no external preregistration is claimed.

12 Future Work and Research Evaluation Plan

The central hypothesis is that a hierarchy with explicitly controlled shapes and placement can offer useful generation quality at favorable measured accelerator cost. This hypothesis remains open. The following experiments would separate representation quality, prediction quality, and systems behavior.

Tokenizer study. Fix a corpus, token vocabulary, byte-expansion protocol, and held-out split before tuning. Compare a single-grid RVQ against multiple level schedules at matched code capacity and parameter budget. Report token reconstruction CE, token and sequence exact match, per-level usage/perplexity, actual serialized bits including fallback and metadata, and padding overhead. Nominal code capacity $\sum_k n_k \log_2 V$ is not an achieved entropy-coded file size.

Prior study. Evaluate both teacher-forced code loss and free-running generation. Compare shared versus independent level parameters, joint versus frozen-tokenizer training, and clean versus perturbed earlier-level conditioning. Fix decoding rules, output lengths, and seeds. An incorrect early prediction can affect all later contexts, so per-level teacher-forced accuracy alone is insufficient.

Objective and phase ablations. Sweep λ_H, λ_C, temperature, entropy weights, dropout probability, and phase duration, including zero-regularization controls. Measure whether marginal diversity improvements survive in hard assignments and decoded text. Compare codebook freezing with continued joint EMA and measure encoder drift while coarse codebooks are fixed.

TPU systems study. Record TPU generation, host count, mesh dimensions, runtime versions, precision, batch sizes, and all sequence shapes. Report compilation time separately from synchronized steady-state throughput and latency; include peak memory, collective costs, useful-token fractions, and checkpoint/input overheads. Carry the completed CPU comparison of padded versus actual-length/sparse execution to TPU, and investigate more efficient distance kernels. Sweep `dp`/`fsdp` meshes and compare float32 with appropriate mixed-precision alternatives.

Matched end-to-end baselines. Compare against token-autoregressive Transformers, diffusion language models, and the original NCM implementation under matched datasets, model capacities, training budgets, and generation protocols. Report quality–cost curves rather than a single speed ratio. Use multiple independent seeds where feasible and report uncertainty and failures. Toy training reconstruction is not a held-out language threshold or a matched quality baseline. Prioritize a fixed real-corpus protocol before larger models.

13 Discussion and Limitations

The completed CPU milestones make the reference program runnable and resumable. The collision ceiling identified a representation bottleneck that decoder-only replay could not repair. Balanced coverage and maintained coarse codebooks addressed that failure on a tiny complete domain. Marginal entropy, conditional TC and output fidelity measure different properties:

improving one cannot stand in for the others. Hard stability passed while reliable alignment still failed. The maintenance package has not been decomposed into causal component ablations.

The 26-byte fixture overfit, and the finite-domain studies share a 16-state support. Neither establishes useful pretrained transfer or competitive language modeling. Different follow-ups used different seed cohorts and samplers; within-protocol paired comparisons are valid, but row-to-row TV changes do not isolate the effect of one intervention. Compiler step estimates verify local arm parity, not total adaptation savings. The SCALE companion [9] formalizes broader hypotheses without presenting them as completed experiments.

No TPU, multi-host throughput or fault-injection recovery was evaluated. Virtual CPU devices test placement semantics, not accelerator interconnects. Atomic local checkpoints and exact resume are implemented, but a distributed fault-tolerant lifecycle remains future work. The generator has no prompt, learned-length or EOS interface. Complete mixed-symbol byte serialization, fallback selection, vocabulary-sharded lookup and Rust orchestration are open. Actual-length inference reduces the padded cost locally; large-vocabulary distance and dense attention costs still need investigation.

The original NCM authors retain credit for the method; this is an independent systems adaptation. Sources, exact evidence inputs and synchronized manuscript builds accompany the PDFs. An archival DOI and evaluation on a fixed real corpus would improve reproducibility and scientific reach.

14 Conclusion

The system now includes deterministic data experiments, frozen-codec training, seeded generation, resumable state and measured CPU inference paths in addition to the original functional quantization and SPMD contracts. The recorded 44 tests support software behavior. On the finite A/B domain, 20 new-seed models passed the shared coarse-stability gate, but the reliable teacher-alignment gate failed. The next scientific stages are component ablations, held-out real-corpus quality/cost evaluation and actual TPU measurements, followed by external pretrained-transfer tests where justified.

A Repository Map and Reproduction

The implementation reference is revision `1d600d7`, package version 0.1.0. `paper/source_manifest.sha256` identifies the unchanged implementation files; `paper/validation.json` links the recorded suite to the current evidence. Shared notation, metrics, report-derived tables and bibliography metadata are included in both sources. `paper/README.md` documents builds and the cross-paper audit. Relative paths below are repository paths.

A fresh CPU environment can run:

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -e '.[test]'
pytest -q
python examples/train.py
```

The declared dependency ranges permit versions other than the tested stack; pin the versions listed in Section 11 for closer reproduction. The multi-device pytest entry launches its own CPU subprocess. Both manuscripts use standard LaTeX packages with shared local inputs and generated embedded bibliographies. No BibTeX run or external raster figure is needed; see `paper/README.md` for the two-document build and audit commands.

Table 6: Repository components relevant to the manuscript. Paths are relative to the repository root.

Path	Responsibility
`src/ncm/config.py`	Immutable shape, Transformer, and loss configuration
`src/ncm/grids.py`	Area-pooling and interpolation matrices; mask normalization
`src/ncm/rvq.py`	Quantizer scan, sufficient statistics, explicit EMA
`src/ncm/tokenizer.py`	Token codec, byte embedding selection, separate decoder API
`src/ncm/generator.py`	Shared Transformer scan and seeded generation
`src/ncm/losses.py`	Normalized reconstruction, commitment, entropy, concept, and CE terms
`src/ncm/training.py`	Joint model, complete training state, optimizer/EMA step
`src/ncm/sharding.py`	Mesh, placement, process-local input helper, compiled contract
`src/ncm/metrics.py`	Exact match, hard usage, and perplexity utilities
`src/ncm/coarse.py`	Deterministic warmup, hard usage and centroid maintenance
`src/ncm/experiment.py`	Byte experiments and atomic checkpoints
`src/ncm/scale_experiment.py`	Paired finite-domain teacher studies
`reports/`	Frozen protocols, metrics and benchmark evidence
`tests/`	Unit, integration, and virtual multi-device checks
`examples/train.py`	Three-step synthetic execution example

B Global Training Transition and Resume Contract

```
# Schematic: exact implementation is in training.py and coarse.py.
# Warmup uses initial training batches before the saved initialized step 0.
def step(state, batch, phase, key):
    loss, diagnostics, grads, stats = objective_and_grad(state, batch, key)
    params, neural_state = update_neural(state, grads)
    books, coarse_state = maintain_codebooks(state, stats, phase, key)
    candidate = pack(params, neural_state, books, coarse_state, state.step + 1)
    return select_global_nonempty(batch.mask, candidate, state), diagnostics
```

Checkpoint state retains configuration and data hashes, step-addressed batch selection, serialized random-key state, codebooks, optimizer state, warmup flags, usage/replacement counts and schedule progress. The initialized step-zero checkpoint is persisted, and resume does not rerun warmup. Frozen prior and decoder replay checkpoints retain their respective stage and source identities. Bit-exact same-backend replay and compatibility with a legacy 500-step checkpoint are part of the recorded verification; cross-platform bit identity is not asserted.

References

[1] Hermann Kumbong, Ishan S. Khare, Iddah Mlauzi, Andrew Woen, Charles Ding, E. Kelly Buchanan, Simran Arora, Azalia Mirhoseini, and Christopher Ré. Next-Concept Modeling: Fast, Parallel Coarse-to-Fine Text Generation. *arXiv preprint arXiv:2610.14162*, 2026. https://arxiv.org/abs/2610.14162.

[2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs. Software, 2018; version 0.10.2 used in this work. https://github.com/jax-ml/jax.

[3] Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. Flax: A neural network library and ecosystem for

JAX. Software, 2020; version 0.12.8, Linen API, used in this work. https://github.com/google/flax.

[4] Flax Authors. Linen transformations: `flax.linen.scan`. https://flax-linen.readthedocs.io/en/latest/api_reference/flax.linen/transformations.html. Accessed November 12, 2026.

[5] OpenXLA Project. XLA documentation. https://openxla.org/xla. Accessed November 12, 2026.

[6] Fabian Balensiefer. Next-Concept Modeling for TPU. Public software repository, package `tpu-ncm`, version 0.1.0, implementation and report snapshot `1d600d7`, November 12, 2026. https://github.com/fbalensiefer/Next-Concept-Modeling-for-TPU/tree/1d600d7f4598ddb7e2808b29ee8f6eefe01cfb78.

[7] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural Discrete Representation Learning. *Advances in Neural Information Processing Systems*, 30, 2017. https://arxiv.org/abs/1711.00937.

[8] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An End-to-End Neural Audio Codec. *arXiv preprint arXiv:2107.03312*, 2021. https://arxiv.org/abs/2107.03312.

[9] Fabian Balensiefer. SCALE: Dependency-Aware Representation Compilation from Pretrained Transformers to Hierarchical Generative Models. Companion research manuscript with finite-domain mechanism studies, revised November 12, 2026. https://github.com/fbalensiefer/Next-Concept-Modeling-for-TPU/blob/main/output/pdf/SCALE_research_paper.pdf.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30, 2017. https://arxiv.org/abs/1706.03762.

[11] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. *arXiv preprint arXiv:2006.16668*, 2020. https://arxiv.org/abs/2006.16668.

[12] JAX Authors. `jax.sharding` module: Mesh, NamedSharding, and PartitionSpec. https://docs.jax.dev/en/latest/jax.sharding.html. Accessed November 12, 2026.

[13] JAX Authors. Manual parallelism with `shard_map`. https://docs.jax.dev/en/latest/201/shard-map.html. Accessed November 12, 2026.

[14] David Arthur and Sergei Vassilvitskii. k-means++: The Advantages of Careful Seeding. *Proceedings of the ACM–SIAM Symposium on Discrete Algorithms*, 2007, pp. 1027–1035. https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf.